

نسخة من المحادثة

مراجعة هندسية لمشروع Sui Arbitrage Bot

1 يوليو 2026

المستخدم

لو عاوزك تروح لمسار معين في اللاب توب تروح ازاي

المساعد

ابعت لي المسار كاملا، مثل:

```
~/Users/web3eg/Documents/sui/project`
```

وسأنفذ الأوامر داخله بتحديد كملد العمل، بما يعادل:

```
cd /Users/web3eg/Documents/sui/project
```

حاليا أعمل داخل:

```
~/Users/web3eg/Documents/sui`
```

أستطيع قراءة الملفات عموما، لكن الكتابة خارج المسارات المسموح بها قد تتطلب موافقتك.

المستخدم

```
/Users/web3eg/Desktop/sui-arbitrage-bot
```

المساعد

تمام، المسار المطلوب هو:

```
~/Users/web3eg/Desktop/sui-arbitrage-bot`
```

قل لي ما الذي تريد تنفيذه داخله. أستطيع قراءته، لكن تعديل ملفاته قد يتطلب موافقتك لأنه خارج مساحة العمل الحالية.

المستخدم

شوف المشروع دا عبارته عن بوت بس مفيش فيه ارباح خالص مش عارف ليه عاوزك تفصل المشروع وتطلع الاسباب

المساعد

منطق اكتشاف وتنفيذ الأربيتراج، الرسوم والانزلاق السعري، ثم أشغل الاختبارات أو المحاكاة المتاحة بدون تنفيذ صفقات حقيقية. تمام. هعمل تشخيص قراءة فقط للمشروع: أفهم المعمارية،

المساعد

حديثه. هركز الآن على مقارنة "الربح المتوقع" بما حدث فعلا في آخر تشغيل، لأن الفجوة بينهما غالبا هي مكان العطل الحقيقي. المشروع واضح إنه متعدد الطبقات: عقود Move، ماسح/منفذ Rust، وتقارير تشغيل فعلية

المساعد

في الدراسة الورقية. سأثبت الآن هل المشكلة سوقية فقط أم أن إعدادات البحث نفسها تستبعد الفرص التي بني عليها توقع الربح. أعاد صفر مرشح باستمرار. كمان الRPC كان يرد بـ429 ويسقط بعض مجمعات Turbos، وهي تحديدا من أهم مصادر الربح صفقات؛" هو لم يصل أصلا لمرحلة تجربة أي صفقة. خلال نحو 35 دقيقة كان عنده حتى 419 مجمعا، لكن فلتر المرحلة الأولى ظهرت نقطة مهمة: البوت لم "يخسر

المساعد

التشخيص المختصر

البوت لم يحقق أرباحاً لأنه لم ينفذ أي صفقة أصلاً. آخر تشغيل استمر 35 دقيقة وكانت النتيجة:

- 419 مجعاً محملاً.

- صفر مرشحين.

- صفر Dry-runs.

- صفر معاملات.

- الربح والخسارة: صفر.

هذا مثبت في [تقرير التشغيل](Users/web3eg/Desktop/sui-arbitrage-bot/runs/report-20260630-225337.md).

الأسباب الرئيسية

1. فلتر البحث يمنع الوصول إلى التنفيذ

لذلك لا توجد فرصة للتحقق الرسمي من المسارات القريبة من الربحية.

الربح المحلي الحد الأدنى مسبقاً، في [main.rs](Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/main.rs:229).
الماسح لا يرسل المسار إلى Dry-run إلا إذا تجاوز

2. خطأ في تطبيق حد الربح بالدولار

التحويل للدولار يستخدم لاحقاً فقط داخل مسار owned execution، والذي لم يصل إليه البوت أصلاً.

config.min_profit مباشرة في [main.rs](Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/main.rs:235).
المشروع يحسب effective_min_profit_mist () لحد 0.05\$، لكن الماسح يستخدم

3. نسخة التشغيل لا تغطي نفس السوق المستخدم في تقرير الأرباح

تقرير day/17\$ اعتمد على:

- Cetus

- Turbos

- Bluefin

- Momentum

- Kriya

وبالتالي تتم مقارنة نتائج نظام محدود بدراسة ورقية أوسع.

وTurbos فقط، كما يظهر في [config.rs](Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/config.rs:228).
أما البوت الحي فيفهرس وينفذ على Cetus

4. فلترة المجمعات عدوانية وغير موحدة

هذا قد يستبعد مجموعات صغيرة لكنها تحتوي على انحراف سعري مربح.

لقيمة دولارية موحدة في [indexer.rs](Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/indexer.rs:357).
أي مجمع تحت قيمة سيولة خام ضخمة، ثم تحتفظ بأعلى 300 فقط لكل منصة. المقارنة تتم بقيمة CLMM خام دون تحويلها
الإعدادات تستبعد

5. مزود RPC لا يتحمل الحمل

وبعض المجمعات تم إسقاطها بالكامل. وهذا مؤثر لأن تقرير الأرباح نفسه يقول إن Cetus↔Turbos هو أهم مصدر للربح.
السجل يحتوي على ردود 429 متكررة أثناء تحميل Turbos ticks،

6. الربح الورقي غير قابل للتحصيل كما هو

وثائق المشروع نفسها تقول إن:

- الربح الورقي كان نحو \$17/day خلال ساعات نشطة فقط.
- بعد التأخير والمنافسة والغاز يصبح غالبا أقل من \$2/day.
- معظم الفرص الصغيرة تصبح سالبة بعد خسارة السباق.
- التوصية المكنوبة حاليا هي NO-GO.

7. مدة التشغيل غير كافية للحكم الاقتصادي

من 17 ساعة موزعة على فترتين. لكن حتى مع قصر المدة، صغر مرشحين يكشف مشكلة تغطية وفلتر، وليس مجرد سوء حظ. التشغيل الحقيقي كان 35 دقيقة فقط، بينما الدراسة الورقية اعتمدت على أكثر

سلامة الكود

68 Rust: اختبارا نجحوا بالكامل. إذا المشكلة ليست انهيارا برمجيا ظاهرا، بل تصميم البحث، التغطية، البيانات، والجدوى الاقتصادية. شغلت اختبارات

الأولويات المقترحة للإصلاح

1. استخدام effective_min_profit_mist () داخل الماسح.
 2. تسجيل عدد الدورات المكتشفة وأفضل ربح قبل الفلتر، بدل إظهار candidate=false فقط.
 3. تمرير أفضل المسارات القريبة لـ Dry-run حتى لو لم تتجاوز الفلتر المحلي.
 4. توحيد السيولة بالدولار وعدم ترتيبها بقيم CLMM الخام.
 5. إصلاح Turbos tick loading وتقليل طلبات RPC والـ 429.
 6. مطابقة منصات التشغيل مع منصات دراسة الأرباح.
 7. تشغيل Paper mode لمدة 24-72 ساعة قبل المخاطرة بأي أموال.
- التشغيل يحتوي رابط QuickNode كاملا مع مفتاح الوصول. أنصح بتغيير المفتاح فورا وعدم تسجيل رابط RPC كاملا مستقبلا. ملاحظة أمنية مهمة: سجل

المستخدم

pwd

المساعد

```
~/Users/web3eg/Desktop/sui-arbitrage-bot`
```

المستخدم

I do NOT want any code changes yet.

Do NOT modify anything.

Do NOT create commits.

Do NOT push.

Do NOT edit .env.

Do NOT run live trading.

Your role is to become an expert on this repository first.

I want you to spend your first session only studying and understanding the project.

Your goal is to understand the entire architecture before making a single suggestion.

Please perform a complete engineering review.

Study:

- Overall architecture
 - Rust code
- Move packages
 - PTB builder
- Flash loan flow
- Owned-capital flow
 - Scanner
- Quote engine
- Tick ingestion
- Route generation
 - Executor
- Dry-run pipeline
- Submit pipeline
- Risk management
 - Metrics
- Configuration
- Dashboard
 - Logs
- Documentation
 - README
- Previous reports
- Recent commits
- Current branch

Then produce a detailed report.

The report should include:

1. High-level architecture

Explain the complete system from scanner to execution.

2. Execution pipeline

Explain every stage.

Candidate

↓

Quote

↓
Requote
↓
Dry-run
↓
Submit
↓
Settlement
↓
Metrics

3. Repository quality

What is good?
What is bad?
What is overly complex?
What is elegant?

4. Technical debt

List every important technical debt.

Rank:
Critical
High
Medium
Low

5. Current blockers

What currently prevents this bot from becoming consistently profitable?

Separate:
Engineering blockers
Architecture blockers
Market blockers

6. Risk review

Would you trust this bot with your own money?

If not:

why exactly?

7. Flash loan design review

Is the flash-loan architecture good?

Would you redesign it?

Why?

8. Owned-capital review

Was moving to owned capital a good idea?

What are the advantages?

What are the disadvantages?

9. Performance review

Where is latency introduced?

Estimate the biggest latency sources.

10. Quote engine review

Do you think the quote engine is robust?

Would you redesign anything?

11. Scanner review

Is the scanner efficient?

Does it scan too much?

Too little?

12. Production readiness

Give a score from 1 to 10 for:

Architecture

Code quality

Maintainability

Reliability

Safety

Performance

Documentation

Production readiness

13. What would YOU do?

If this were your own arbitrage bot...

What would be the first five improvements?

Do NOT implement them.

14. Biggest mistake

What do you think is currently the single biggest design mistake?

Explain why.

15. Final opinion

Imagine you joined this project as Lead Engineer.

Would you continue this architecture?

Would you simplify it?

Would you rewrite major parts?

Would you keep the current strategy?

Be brutally honest.

Important:

This session is READ-ONLY.

I want your engineering opinion.

No code.

No commits.

No fixes.

No implementation.

Only analysis and recommendations backed by evidence from the repository.

المساعد

commit أو push. سأبني التقرير من الكود الفعلي، السجلات، الوثائق، وتاريخ Git، مع فصل ما هو مثبت عما هو استنتاج هندسي. مفهوم تماما: هذه جلسة مراجعة هندسية للقراءة فقط. لن أعدل أي ملف، ولن أشغل تداولاً أو بناء حياً، ولن أنشئ

المساعد

والتسجير والمسح، ثم PTB والتنفيذ والمخاطر، ثم Move، وبعدها الوثائق والتقارير والسجلات وتطابق الادعاءات مع التطبيق. main ومتقدم commit محلياً واحداً عن origin/main، وشجرة العمل نظيفة. سأراجع الآن طبقات النظام بالترتيب: نماذج البيانات الصورة الأولية: الفرع الحالي

المساعد

هو إعادة التسجير الرسمية الوحيدة. سأعامل ذلك كقرار معماري فعلي لا كخطأ تلقائي، ثم أقيم أثره على السرعة والسلامة. و"المسار الحي الحالي": يوجد مكون quoter.rs لإعادة التسجير hop-by-hop، لكن التنفيذ الحالي يتجاوزه ويعتبر Dry-run للـ PTB. حتى الآن ظهر فرق مهم بين "المعمارية الموصوفة"

المساعد

تقرير المراجعة الهندسية الشاملة

تمت المراجعة بالقراءة فقط. لم أعدل ملفات، ولم أشغل تداولاً، ولم أنشئ commit أو push. الحالة الحالية:

- المسار: Users/web3eg/Desktop/sui-arbitrage-bot/

- الفرع: main

- الفرع متقدم commit واحداً عن origin/main.

- آخر commit محلي: 53b8ff5 لإضافة /runs إلى .gitignore.

- شجرة العمل نظيفة.

- آخر تشغيل فعلي كان على commit 594cc72.

مسار بيانات بطيء، إعدادات أمان لا تعمل كما توحى أسماؤها، وتغطية سوقية أضيق من التقارير التي بنيت عليها توقعات الربح. المشكلة ليست "Bug" واحد يمنع الربح، بل فجوة بين منظومة البحث الورقية ومنظومة التنفيذ الحية، إلى جانب الخلاصة قبل التفاصيل: المشروع يحتوي على نواة أمنية جيدة وأفكار هندسية أنيقة، لكنه ليس بوت أريبتراج Production-ready.

1. المعمارية العامة

النظام مقسوم إلى طبقتين.

الطبقة خارج السلسلة — Rust

مسؤولة عن:

1. اكتشاف مجمعات Cetus وTurbos.
2. قراءة حالة المجمعات.
3. تحميل نطاق من ticks لكل CLMM.
4. حفظ الحالة في ReserveCache.
5. بناء graph للعملات والمجمعات.
6. توليد دورات تبدأ وتنتهي بالعملة الأساسية.
7. تسجير المسارات محلياً.

8. اختيار حجم الصفقة.

9. بناء PTB.

10. تنفيذ Dry-run.

11. تطبيق RiskGuard.

12. التوقيع والإرسال إذا كان submit_enabled=true.

13. تسجيل المقاييس والنتائج.

نقطة التشغيل الأساسية موجودة في [76:main.rs](/Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/main.rs).

الطبقة على السلسلة — Move

تتكون من:

- executor.move: بوابة الربح.
- cetus_adapter.move: موحدة swap إلى Cetus تحويل واجهة.
- turbos_adapter.move: موحدة swap إلى Turbos تحويل واجهة.
- amm_v2.move: مرجعي للاختبارات AMM.
- flash.move: مرجعي Flash lender.
- math.move: Rust مطابق لـ V2 حساب.
- admin.move: صلاحيات إدارة المكونات المرجعية.

ينشئ ArbReceipt بلا abilities، ولذلك يجب استهلاكه داخل PTB نفسها. عند التسوية:
أهم جزء هو [executor.move](/Users/web3eg/Desktop/sui-arbitrage-bot/sources/executor.move).

```
final_amount >= initial_amount + min_profit
```

والا تفشل المعاملة كلها ذربا.

هذا التصميم بسيط وأنيق وقوي أمنيا.

مسارات التمويل

يوجد مساران:

- بوابة الربح → سداد → تحويل المتبقي → swaps → اقتراض: Flash.
 - بوابة الربح → إعادة الناتج للمحفظة → swaps → GasCoin من SUI فصل: Owned.
- يوجد أيضا مسار Liquidation تجريبي مبني فوق Pyth و Scallop، لكنه لم يصل لنفس درجة الإثبات التشغيلي لمسار الأربيتراج.

2. خط التنفيذ الكامل

Candidate

افتراضيا يأخذ البرنامج نسخة كاملة من cache، ثم يعيد بناء adjacency graph ويبحث عن دورات بسيطة تبدأ وتنتهي بـ SUI. كل 500ms

المسح في [117:scanner.rs](/Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/scanner.rs):

- يدعم دورات من 2 إلى max_hops.
- القيمة الحالية الافتراضية لـ max_hops هي 3.

- يجرب شبكة أحجام، ثم ternary search.

- يطرح gas estimate و flash fee.

- يعيد أفضل فرصة واحدة فقط.

المشكلة: كثير من المسارات المربحة تاريخيا كانت 4 hops أو أكثر، بينما الإعداد الجي هو 3.

Quote

التسعير المحلي يستخدم:

- amm.rs للمجمعات V2.

- clmm.rs لمجمعات Cetus و Turbos.

Q64.64. إذا تجاوز حجم الصفقة نطاق ticks المحمل، يرفض التسعير بدل اختراع سيولة غير معروفة. هذا قرار محافظ وجيد.
محرك CLMM يعبر ticks ويستخدم U256 وحساب

Requote

لتنفيذ quote رسمي عبر devInspect، لكنه غير مستخدم في المسار الحالي.
يوجد [quoter.rs](/Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/quoter.rs)

الوحيد. هذا يوفر 300-500ms تقريبا، لكنه يعني أن min_out يبنى من التسعير المحلي وليس من quote رسمي حديث.
التنفيذ الحالي يعتبر Dry-run للPTB الكاملة هو authoritative requote

أيضا:

- موجود لكنه غير مستخدم ARB_MAX_QUOTE_AGE_MS -

- staleness موجود لكنه غير مستخدم كحارس PoolState.last_seq -

- لا يوجد timestamp فعلي للquote.

Dry-run

يبنى البرنامج نفس PTB المراد إرسالها، ثم يستدعي dry_run_transaction_block.

يستخرج:

- حالة النجاح أو الفشل.

- استهلاك الغاز.

- تغير رصيد العملة الأساسية للمرسل.

هذه أقوى نقطة في المسار خارج السلسلة: Dry-run يفحص PTB الحقيقية وليس نموذجا منفصلا.

فلتر scanner الأول، ثم يجرب الأحجام من الأكبر إلى الأصغر ويقبل أول حجم ناجح، وليس الحجم صاحب أعلى net profit.
لكن في الوضع Owned لا يصل البرنامج لهذه المرحلة إلا إذا اجتاز المسار

Submit

بعد نجاح Dry-run:

1. يستدعي RiskGuard.

2. إذا كان الإرسال مفعلا، يحمل keystore.

3. توقع نفس TransactionData.

4. ترسل باستخدام WaitForLocalExecution.

5. تقرأ balance changes.

6. يسجل realized P&L.

المسار في [executor.rs](/Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/executor.rs:934).

لا توجد إعادة Dry-run أو فحص quote age بين Dry-run والإرسال.

Settlement

في Owned:

```
split GasCoin
→ executor::begin
→ swaps
→ executor::settle
```

في Flash:

```
borrow
→ executor::begin
→ swaps
→ settle and return
→ repay
→ transfer remainder
```

إذا لم يتحقق الربح المحدد، تفشل PTB بالكامل وتكون الخسارة غارا فقط.

Metrics

يوجد نظام Prometheus خفيف بدون dependencies ثقيلة، مع:

- Latency مرحلة لكل.
- Latency لكل RPC.
- candidates.
- reject reasons.

```
- dry-run success/failure.
- submitted/failed.
- predicted/realized profit.
```

- cache size.
- pools.
- obligations.
- RPC health.

التصميم خفيف وأنيق، والـ Dashboard جيد كبدية، لكنه يعرض مقاييس غير موصولة فعليا مثل Sizing وبعض reject reasons.

3. جودة المستودع

الجيد

- بوابة الريح في Move ممتازة وبسيطة.
- الفصل بين executor و DEX adapters جيد.
- واحدة ذرية هي الاختيار الصحيح PTB -
- حساب V2 مطابق بين Rust و Move.
- محرك Cetus حقق parity كاملة في التقرير: 1,034/1,034.
- رفض التسعير خارج نطاق ticks أفضل من over-quoting.
- الحقيقة قرار قوي لـ Dry-run PTB.
- المتكررة RPC calls يقلل ObjRefCache -
- استخدام feature gate لمسار Sui SDK يحافظ على خفة الاختبارات الأساسية.
- الوثائق صريحة نسبيا بشأن NO-GO الاقتصادي.
- المقاييس منخفضة التكلفة.
- يوجد وعي جيد بمخاطر المنافسة والغاز والـ MEV.

السيئ

- الوثائق لا تعكس التنفيذ الحالي دائما.
- منفصلا رغم تجاوزه في المسار الحي authoritative requote ما زال يصف README -
- أرقام الاختبارات في README قديمة.
- يوجد عدد كبير من knobs غير المستخدمة أو مستخدمة جزئيا.
- البحث الورقي ومنظومة التنفيذ الحية لا تستخدمان نفس الvenues ولا نفس عدد hops.
- بعض المكونات موجودة كـ scaffolding أكثر من كونها production code، مثل Backrun و DeepBook.
- واحدة struct في indexing و signing و Pyth و liquidation و arbitrage أصبح كبيرا جدا ويجمع Config -
- أخطاء RPC تؤثر في graph بدون lifecycle واضح لحذف الحالة القديمة.
- السجل يطبع رابط RPC كاملا بما فيه credential.

المعقد أكثر من اللازم

- جمع arbitrage و liquidation و oracle refresh و flash و owned mode في executable واحدة.
- معقدة جدا قبل إثبات اقتصاديات transaction في Pyth/Wormhole/Scallop/swap/profit gate تجمع PTB Liquidation - liquidation.
- ResolvedHop فعلي تضيف طبقة يمكن أن تتحرف عن builder ثم إعادة تحويلها إلى IPtbStep الخاصة بـ abstraction -
- وجود quoter.rs كامل بينما المسار الحي لا يستخدمه يزيد الالتباس.

الأنيق

- Hot-potato receipts.
- Stateless executor.
- Adapter convention.
- مشاركة lowering نفسها بين Flash و Owned.
- ObjRefCache.
- CLMM في محرك Depth-aware refusal.
- Atomic Dry-run ثم settlement gate.

Critical

1. إعداد ARB_FLASH_ENABLED غير مستخدم فعليا.
مسار Flash يحدده execution_mode، وليس flash_enabled. هذا يجعل kill/config switch مضللا.
2. ARB_MAX_QUOTE_AGE_MS غير مطابق
لا يوجد حارس حقيقي لعمر الحالة قبل الإرسال.
3. RiskGuard يستخدم سعر SUI 1\$
وبالتالي daily-loss accounting بالدولار غير صحيح.
في [main.rs](/Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/main.rs:102)،
4. حد الربح بالدولار لا يطبق في scanner.
في config.min_profit يستخدم scanner، لكن Owned executor يستخدم داخل effective_min_profit_mist()
route يمكن للمرحلة الأولى إسقاط. (main.rs)(/Users/web3eg/Desktop/sui-arbitrage-bot/offchain/src/main.rs:229).
قبل الوصول إلى الحد الصحيح
5. المجمعات التي خرجت من active set لا تحذف من cache أو registry.
هذا يسمح ببقاء حالة قديمة لمسارات لم تعد نشطة. upsert يقوم فقط بـ sync_pools
6. تسريب credential داخل السجل.
أول سطر من live log يحتوي رابط QuickNode كاملا مع token.

High

1. بيانات المجمعات تحدث كل 10 ثوان بينما الأريتراج يحتاج sub-second state.
2. 101 استجابة 429 و 21 عملية إسقاط tick ظهرت في تشغيل واحد.
3. Turbos tick prices باستخدام مشتقة f64 مطابقا للبروتوكول liquidity الخام، وليس قيمة أو depth موحد بالدولار.
4. ترتيب السيولة بين المجمعات يعتمد على liquidity الخام، وليس قيمة أو depth موحد بالدولار.
5. max_hops=3 يستبعد عددا معتبرا من المسارات الموجودة في تقارير الربح
6. التشغيل الحي يدعم Cetus و Turbos فقط، بينما P&L الورقي اعتمد خمسة venues.
7. Owned mode يختار أول حجم ناجح من الأكبر، وليس الحجم الأعلى ربحا
8. daily loss داخل الذاكرة فقط؛ إعادة تشغيل العملية تصفر RiskGuard
9. لا يوجد circuit breaker مبني على RPC degradation أو stale cache أو نسبة 429.
10. لا يوجد تحقق صريح أن Owned mode يستخدم SUI فقط رغم أن base_token يبدو عاما.
11. Scallop من authoritative يدويا بدل قراءة configuration يأتي من Flash fee.
12. زائد بدل فشل صريح على اختلاف الأطوال mismatch ويمكنه تجاهل zip يستخدم lower_swaps

Medium

- إعادة بناء graph والدورات كل 500ms رغم أن الحالة تتغير كل 10s.
- snapshot في كل ticks و PoolState كامل لكل cloning -
- يعيد أفضل فرصة واحدة فقط scanner -
- لا يوجد dedup/cooldown للفرص المستمرة.

- لا يوجد priority queue للأحداث أو dirty-pool incremental scan.
- بعض metrics معرفة لكن غير مسجلة فعليا.
- health model وليس RPC، مجرد آخر نتيجة rpc_up -
- SLOs أو alerts لا يحتوي dashboard -
- توجيه بوظائف غير موجودة max_quote_age_ms و wws_url و flash_enabled -
- موجود لكن غير موصول بمسار التشغيل BackrunSource -
- DeepBook adapter يعيد Dex::Cetus كplaceholder -
- خطأ indexer task يسجل ثم يستمر scanner على cache قديمة بدل إيقاف الإرسال.
- فشل الصريح إذا كان المسار خاطئا keystore قد ينشئ submit path في FileBasedKeystore::load_or_create -

Low

- تعليقات و README قديمة.
- أسماء ws.rs لم تعد دقيقة لأن التنفيذ polling.
- تكرار منطق build/dry-run.
- typed ليس مقسما إلى أقسام Config -
- لا توجد schema validation شاملة عند startup.
- بعض الوثائق تصف خططا مستقبلية بصيغة توجيه بأنها منفذة.

5. العوائق الحالية أمام الربحية

عوائق هندسية

- بيانات متأخرة حتى 10 ثوان.
- RPC rate limiting.
- تحميل Turbos ticks مكلف.
- لا يوجد staleness gate.
- mismatch في min-profit.
- محدودة venues.
- محدود hop limit.
- لا تنظف العناصر القديمة active cache.
- قياس المخاطر بالدولار غير صحيح.
- لا توجد بيانات فعلية كافية عن react→dry-run→submit→land.

عوائق معمارية

- الاعتماد على JSON-RPC polling remote لبوت ينافس على micro-spreads.
- مسح graph واسع دوريا بدل الاستجابة لتغيرات مجموعات محددة.
- وضع Dry-run كأول authoritative validation بعد فلتير محلي صارم.
- دمج research strategies متعددة داخل runtime واحد قبل تثبيت استراتيجية مربحة.
- محاولة جعل execution pipeline عاما جدا بينما universe الحي صغير وغير مطابق للبحث.

عوائق السوق

- تقرير حديث لمدة 10.25 ساعة قدر الربح الورقي بـ\$1.10/day فقط.
- بعد الاحتكاكات أصبحت النتيجة المتوقعة -\$0.59/day.
- الفرص مركزة جدا ومتعلقة بالتقلب.
- كان أقل من تكلفة محاولة فاشلة median profit.
- المنافسون أسرع وعلى الأرجح validator-adjacent.
- لا توجد mempool عامة.
- أفضل تقرير قديم \$17/day كان خلال نافذة نشطة غير ممثلة لليوم الكامل.
- حتى هذا الرقم انخفض إلى أقل من \$2/day بعد نموذج الاحتكاكات.

6. مراجعة المخاطر

لا، لن أثق به بأموالي حالياً.

ليس لأن Move executor ضعيف؛ بل لأن السلامة التشغيلية غير مكتملة:

- إعدادات أمان موجودة لكنها لا تعمل كما توجي الأسماء.
 - سعر الدولار داخل RiskGuard خاطئ.
 - حد الخسارة يصفر عند restart.
 - لا يوجد stale-state circuit breaker.
 - يستمر scanner قد يتوقف بينما indexer -
 - سجل بنص واضح credential.
 - استخدم قبل اكتمال القياس submit_enabled=true -
 - لا يوجد canary sizing أو progressive exposure.
 - لا يوجد إثبات حي لصفقات Cetus/Turbos ناجحة وربحية بعد الهبوط على mainnet.
 - لا يوجد fail-closed شامل عند ضعف RPC.
- principal نتيجة swap غير مربحة، لكن لا أثق بأن النظام لن يحرق الغاز، يخطئ في accounting، أو يعمل على state قديمة. أثق نسبياً بأن بوابة Move تمنع فقدان

7. مراجعة Flash Loan

الهيكل الأساسي جيد جداً:

```
borrow
→ begin
→ swaps
→ settle and return
→ repay
→ transfer profit
```

وجود receipt ين hot-potato يجعل التدفق آمناً ذرياً.

لكنني سأعيد تصميم الجزء التشغيلي، وليس المبدأ:

- قراءة flash fee من البروتوكول.
- إزالة flash_enabled أو جعله gate حقيقياً.
- فصل provider configuration عن liquidation configuration.
- التحقق من package/version عند startup.

- إضافة provider health readiness.
- منع submit إذا لم تحل جميع object references مسبقاً.
- قياس كلفة shared-object contention.

Flash loan يزيد:

- عدد shared objects.
 - احتمالات version mismatch.
 - PTB complexity.
 - gas.
 - زمن البناء.
 - احتمالات فشل خارج DEX نفسها.
- مع رأس مال يقارب SUI 185، لا أرى Flash Loan خياراً افتراضياً جيداً قبل إثبات أن الحجم الأكبر يزيد net P&L فعلاً.

8. مراجعة Owned Capital

الانتقال إلى Owned Capital كان قراراً جيداً للاختبار والإثبات الأولي.

المزايا

- إزالة اعتماد Scallop من مسار الأربيتراج.
- أبسط PTB.
- أقل gas.
- عدد shared objects أقل.
- أخطاء package/version أقل.
- أقل latency.
- تشخيص failures أسهل.
- مناسب لرأس المال الحالي.

العيوب

- رأس المال محدود.
 - يحتاج coin selection و wallet balance RPC.
 - يستخدم GasCoin كرأس مال، مما يعقد إدارة الغاز.
 - التنفيذ الفعلي يعمل منطقياً مع SUI فقط.
 - الحجم يختار من ladder ثابت.
 - البحث يبدأ أصلاً بأحجام مختلفة عن Owned ladder.
 - اختيار أكبر حجم ناجح لا يعني أفضل ربح.
 - overhead قد يضيف wallet fragmentation.
- كالمسار الوحيد حتى إثبات ربح حي مستقر، ثم أضيف Flash فقط للفرص التي تثبت أن capital constraint هو العامل المحدد. قرارى: كنت سأبقى Owned

9. مراجعة الأداء

أكبر مصادر latency مرتبة:

1. تأخر البيانات: 0-10 ثوان

بسبب refresh interval للخوادم. هذا أكبر من كل زمن الحساب الداخلي مجتمعاً.

2. Turbos tick enumeration: ثوان إلى عشرات الثواني

يتم تعداد dynamic fields التي تشمل Positions، وظهر ذلك بوضوح في السجل.

3. RPC throttling/backoff: حتى عدة ثوان

jitter بالإضافة إلى ms 2250 ثم ms 750 هي تقريبا ms 250 retry delays.

4. Dry-run: 100-500 ms غالباً

يعتمد على موقع RPC وحمله.

5. Submit + local execution: 0.5-3 ثوان

حسب الشبكة والcheckpoint.

6. Owned sizing: عدة Dry-runs متسلسلة

يمكن أن يضرب latency بعدد الأحجام.

7. PTB object resolution في أول فرصة

بعض الـ shared objects تحتاج RPC قبل أن يدخل cache.

8. المسح المحلي

غالباً ليس المشكلة الأساسية، لكنه يعاد بلا داع كل 500ms.

التشغيل الأخير احتاج قرابة 2 دقيقة و40 ثانية للوصول من صفر إلى نحو 404 مجمعات، ثم أعاد تحميل ticks كل دورة اكتشاف.

10. مراجعة Quote Engine

Cetus

قوي جداً حسابياً، والدليل التجريبي ممتاز:

- قابلة للتقييم quote 1,034 -

- حتى 190 tick crossing.

- حسب التقرير zero mismatch -

Turbos

أقل ثقة:

- التقرير القديم أظهر أخطاء تصل إلى 952% بدون ticks.

- التحميل الجديد يحل depth جزئياً.

- 64f مشتقة بـ sqrt prices.

- لم أر تقرير parity جديداً يثبت محرك Turbos بعد commit تحميل ticks.

- قبل اختيار النافذة dynamic fields يحدد كل loader.

- قديمة cache تسقط مجمعات أو تعيد errors -

ما الذي سأغيره تصميميا؟

- استخدام tick math integer مطابق لـTurbo.
 - إثبات parity جديد بعد tick ingestion.
 - dynamic fields بدل تعداد كل bitmap/tick_map traversal.
 - snapshot لكل checkpoint و timestamp.
 - نوع نتيجة quote يميز:
 - Valid
 - Stale
 - DepthLimited
 - MissingTicks
 - RPCDegraded
 - عدم تمثيل كل failures بـNone.
- المحرك الرياضي جيد، لكن data acquisition المحيطة به ليست robust بما يكفي.

11. مراجعة Scanner

الماسح مناسب لمئات المجموعات و3 hops من ناحية CPU، لكنه لا يبحث في المساحة الصحيحة اقتصاديا.

يمسح أكثر مما ينبغي

- يعيد بناء graph كل 500ms.
- يعيد فحص كل الدورات حتى لو لم تتغير المجموعات.
- يعمل أثناء bootstrap وعلى cache جزئية.
- لا يستخدم dirty set أو affected-token subgraph.

يمسح أقل مما ينبغي

- max_hops=3.
- فقط Turbo وCetus.
- واحدة base token.
- يعيد مرشحا واحدا فقط.
- لا يحتفظ بأفضل N للـauthoritative validation.
- لا يوجد event-triggered backrun فعلي.
- raw liquidity قد تستبعد مجموعات مهمة بسبب filters.

مشكلة optimizer

عدة CLMM profit curve ticks. قد يحتوي kinks وتغيرات سيولة متعددة. Ternary search قد يعمل غالبا، لكنه ليس إثباتا عاما. الافتراض أن منحنى الربح unimodal غير مضمون عند عبور

12. تقييم الجاهزية

| الجانب | التقييم |

```
| Architecture | 7/10 |
| Code quality | 7/10 |
| Maintainability | 5/10 |
| Reliability | 4/10 |
| Safety | 6/10 |
| Performance | 3/10 |
| Documentation | 7/10 |
| Production readiness | 3/10 |
```

السلامة حصلت على 6 بسبب قوة atomic settlement، لكنها ليست أعلى بسبب ضعف الضوابط التشغيلية.

13. أول خمس تحسينات كنت سأقوم بها

بدون تنفيذ:

1. تثبيت مسار بيانات حي سريع وصحيح
Fullnode/indexer stream staleness gate. مع checkpoints أو مصدر
2. توحيد universe البحث والتنفيذ
venues، نفس pools، نفس hop limits، ونفس sizing بين paper trading وlive runtime.
3. فصل النظام إلى ثلاثة أجزاء
الأساسي مؤقتا runtime من liquidation وإخراج. Market data service، strategy/searcher، execution/risk service.
4. إصلاح طبقة الحماية والتكوين
جعل كل switch حقيقيا، سعر حي، loss state دائم، startup validation، وfail-closed على RPC/indexer degradation.
5. تشغيل shadow/paper pipeline طويل بقياسات حقيقية
عمر البيانات، أفضل edge قبل الفترة، react-to-land estimate، dry-run delta، والفرص المفقودة لمدة عدة أيام متقلبة.
تسجيل كل مرحلة،

14. أكبر خطأ تصميمي

- أكبر خطأ هو بناء استراتيجية micro-spread منخفضة الهامش فوق data plane قائم على polling واسع وبطيء.
- كل العناصر الأخرى تصبح ثانوية أمام ذلك.
- البوت يحاول المنافسة على فرص قد تعيش مئات الملي ثانية، بينما:
- تحديث المجمعات كل 10 ثوان.
 - تحميل Turbos ticks قد يستغرق ثوان.
 - يعيد 429 RPC -
 - بعض ticks تظل قديمة حتى دورة الاكتشاف التالية.
 - بعد ذلك فقط يبدأ Dry-run والإرسال.
- هذا ليس مجرد performance optimization مؤجل؛ إنه تعارض بين طبيعة الاستراتيجية وطبيعة البنية التحتية.

15. الرأي النهائي

لو انضمت كـ Lead Engineer:

- سأحتفظ بـ Move executor.
- سأحتفظ بمبدأ PTB الذرية.
- سأحتفظ بـ Cetus CLMM math بعد parity الممتازة.
- سأحتفظ بفكرة adapters.
- سأحتفظ بـ Dry-run كحارس أخير.

لكنني سأبسط أغلب الطبقة خارج السلسلة.

لن أعيد كتابة المشروع كاملاً، لكنني سأعيد بناء أجزاء كبيرة من:

- data ingestion.
- cache lifecycle.
- scanner triggering.
- configuration.
- risk state.
- execution orchestration.
- observability semantics.

وسأوقف مؤقتاً:

- Liquidation runtime.
- Backrun scaffolding.
- DeepBook scaffolding.
- كمسار افتراضي Flash mode.
- أي إرسال حي.

البيانات تقول إن يوماً هادئاً أعطى $\text{day}/1.10\$$ ورقياً و $\text{day}/0.59\$$ بعد الاحتكاكات. هذا لا يبرر التعقيد أو المخاطرة التشغيلية. أما الاستراتيجية نفسها، فلن أعتبر pure cross-venue arb الحالية مشروعاً تجارياً مثبتاً. أحدث

مريح. النواة تستحق الاستمرار، لكن الاستراتيجية و data plane يحتاجان إعادة تقييم جذري قبل أي محاولة تشغيل مالي جديدة. رأيي الصريح: المشروع أفضل بكثير كمنصة بحث وتسعير و PTB آمنة مما هو كبوت أريتراج

تم استبعاد تعليمات النظام وحجب بيانات دخول RPC الحساسة.